

From Hand-Drawn Schematics to SPICE Netlists: Endpoint-Graph Wire Joining and a Human-Verified Connectivity Benchmark

Bosco Chanam, Chris Dcosta, and Pranavesh Talupuri

Abstract—Automated digitization of scanned analog circuit diagrams into simulation-ready SPICE netlists remains a challenging problem in document understanding. The bottleneck is not component detection or wire extraction but the *joining* step: determining which extracted wire segments connect to which component pins and to each other. Existing approaches use radius-based proximity grabbing, which over-merges components onto shared nets, particularly under the noisy endpoint estimates produced by real detectors. We present (1) an endpoint-graph join model that represents both wire endpoints and component pins as graph nodes with five edge types capturing wire-body, endpoint-endpoint, endpoint-pin, endpoint-to-wire-body (T-junction), and pin-to-wire-body (rail-tap) connectivity; (2) a degree-budget completion algorithm that uses min-cost b-matching to recover dropped connections while structurally bounding over-merge through a per-pin edge budget and self-loop guards; (3) a synthetic evaluation framework with controlled error injection across five severity levels, for robustness analysis independent of ground-truth wire labels; and (4) to our knowledge, the first human-verified net-level connectivity benchmark for hand-drawn circuits (31 images from CGHD-1152), on which we compare join algorithms under the same metric, a comparison no prior image-to-netlist work provides. On 134 scanned images our pipeline achieves wire detection $F1 = 0.976$. On the human-verified net-level benchmark, the best join (degree-budget completion over a high-precision scale-relative endpoint-graph base) reaches connectivity micro- $F1 = 0.890$, versus 0.83 for the prior completion default, 0.67 for the radius-based production join, and 0.62 for the connected-component net-tracing recipe used by recent systems, all evaluated on identical detected wires; it also beats a Hough-plus-proximity line tracer (0.81). Running the join on perfect wire labels does not raise micro- $F1$, showing that connectivity, not wire detection, is the bottleneck. A first-party experiment shows a strong vision-language model reaches 0.923 on the same images, statistically indistinguishable from our join (paired 95% confidence interval includes zero) but at orders-of-magnitude higher cost, with no structural guarantee and non-simulatable output, supporting an explicit geometric model for deployable digitization.

Index Terms—circuit digitization, document understanding, wire joining, graph-based connectivity, SPICE netlist extraction, analog circuit schematics

I. INTRODUCTION

Analog circuit schematics encode functional information (component types, values, and topological connections) in a visual format designed for human interpretation. Converting scanned schematics into machine-readable SPICE netlists enables automated simulation, design verification, and large-scale circuit analysis. While recent advances in object detection have made component recognition increasingly reliable, the *joining* problem, determining which wire segments connect to which component pins and to each other, remains the primary failure mode in end-to-end digitization pipelines.

The challenge is threefold. First, wire endpoints extracted from binarized images are inherently noisy: the same circuit drawn by different drafters produces endpoint positions that vary by 10–80 pixels. Second, components occupy bounding boxes that overlap with wire endpoints, creating ambiguity about whether an endpoint connects to a component’s pin or merely passes nearby. Third, T-junctions, collinear fragments, and crossing wires create complex connectivity patterns that simple radius-based approaches cannot resolve without over-merging.

A natural alternative is to prompt a large vision-language model (VLM) to read the schematic and emit its netlist directly. We test this in Section V-E: a strong VLM attains connectivity accuracy statistically indistinguishable from our geometric pipeline, but it does so at orders-of-magnitude higher cost, returns free-form text rather than a structured netlist, offers no structural guarantee against electrically invalid output (e.g. shorted two-terminal parts), and drops connections on the most complex circuits. These properties motivate an explicit, inspectable geometric pipeline whose output is a netlist by construction and whose accuracy matches the VLM’s while being deterministic and simulatable.

We address these challenges with a two-stage approach: (1) an *endpoint-graph join* model that constructs a graph over both wire endpoints and component pins, with five edge types capturing distinct connectivity patterns; and (2) a *degree-budget completion* algorithm that identifies and re-connects dropped connections using constrained optimization. We evaluate on the CGHD-1152 dataset [1] with a synthetic error injection framework that enables systematic robustness analysis.

B. Chanam and C. Dcosta are with the Symbiosis Centre for Applied AI, Symbiosis Institute of Technology, Pune 412115, India.

P. Talupuri is with the Department of Computer Science, University of Southern California, Los Angeles, CA 90089 USA.

Corresponding author: Bosco Chanam (e-mail: bosco.chanam.btech2021@sitpune.edu.in).

This work was supported by [FUNDING AGENCY AND GRANT NUMBER: fill before submission].

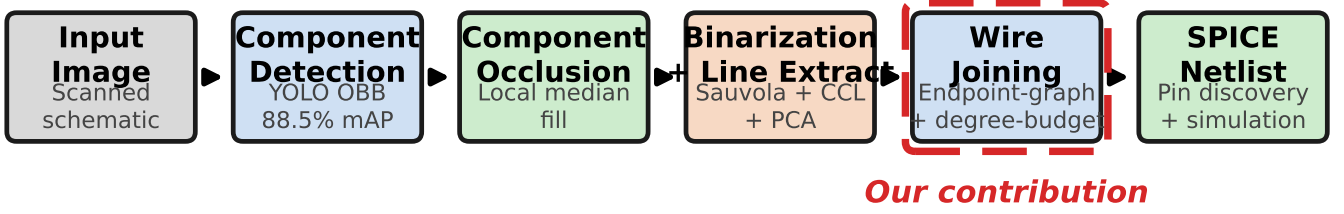


Fig. 1. End-to-end pipeline for scanning circuit diagram digitization. Our contributions focus on the wire joining stage (dashed red), which determines which extracted wire segments connect to which component pins.

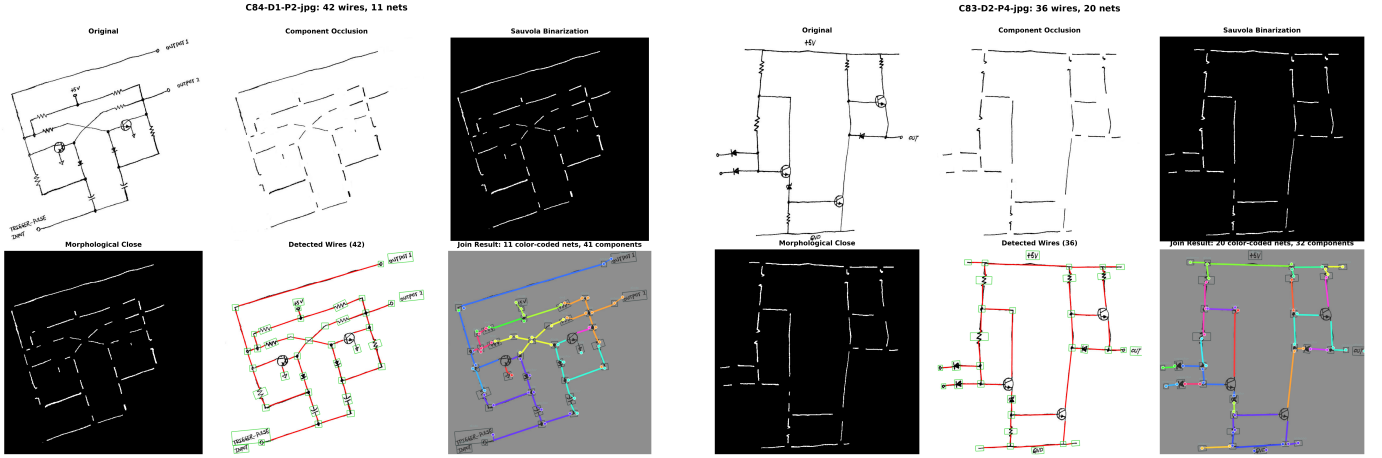


Fig. 2. Pipeline stages on two real hand-drawn circuit images, joined with our `scale_completion` strategy. Left: 42 wires detected, 11 nets recovered; right: 36 wires, 20 nets. Each panel shows, in order: original scan, component occlusion, Sauvola binarization, morphological close, detected wires (red overlay), and the join result (green dots = component pins, grouped into nets).

II. RELATED WORK

A. Circuit Diagram Digitization and Netlist Extraction

The end-to-end conversion of schematic images into machine-readable netlists has progressed rapidly. Kelly and Cole [2] presented a pattern-recognition workflow for digitizing circuit schematics with an 86.4% success rate, combining OCR for component values with network-graph construction. SINA [4], the closest contemporary system, pairs a YOLOv11 detector with connected-component labeling (CCL) and OCR, and invokes a vision-language model (VLM) for reference-designator assignment, reporting 96.47% netlist-generation accuracy, a $2.72\times$ improvement over the prior state of the art. SINA confines the VLM to symbolic labeling and derives nets from CCL proximity rather than trusting the VLM with connectivity, an insight we share (Section II-C). Masala-CHAI [8] (Auto-SPICE) uses large language models to produce SPICE netlists from analog schematics at scale. Most directly comparable to our setting, the concurrent work of Peker et al. [9] (IEEE Access, 2026) builds a fully automated image-to-SPICE pipeline for printed and hand-drawn transistor-level circuits, pairing YOLOv8/11 detection with a *contour-based* node-detection module (components are erased and nets are read from the contours of the residual wire pixels) and, as we do, validates netlists by automated LTspice simulation against a reference. On their in-house seven-topology MOSFET dataset they report 93.33% (printed) and 85.33% (hand-drawn) whole-

netlist success. Their node detection is an instance of the connected-component/contour recipe we evaluate as a baseline (Section V-C); because their dataset (transistor-level analog, authored in-house) and metric (LTspice node-voltage pass/fail) differ from ours (discrete components on CGHD-1152, component-pair F1), the headline numbers are not directly comparable, but the comparison shows contour/CCL node detection remains the prevailing approach, which our endpoint-graph join substantially outperforms on a controlled, identical-input benchmark. Instance-segmentation approaches extract circuit graphs directly from pixels: Bayer et al. [10] segment components and wires for graph extraction over the CGHD lineage. For hand-drawn inputs, object-detection pipelines [12] and the recent JUHCCR-v1 database [11] target component recognition on freehand sketches. Most of these pipelines build on general-purpose object detectors such as the YOLO family [15], [16] and OCR engines such as Tesseract [17], with the ultimate goal of producing simulator-ready descriptions in formats such as SPICE [18]. These works establish detection and recognition as largely solved; the bottleneck shifts to reconstructing electrical connectivity, the focus of our contribution.

B. Datasets and Benchmarks

Progress depends on annotated corpora. The CGHD hand-drawing dataset [1] provides component- and wire-level an-

notations for handwritten schematics. In the analog/mixed-signal (AMS) domain, AMSnet 2.0 [6] contributes a large database built with AI-assisted segmentation for net detection, while AMSBench [7] is a benchmark for evaluating multi-modal LLM capabilities across AMS circuit tasks. Beyond circuits, DiagramNet [5] introduces an end-to-end framework and dataset for non-standard system-level diagrams, including an explicit connection-recognition task scored by F1. We use these benchmarks both as evidence of where current systems succeed (component recognition) and where they fail (connectivity), and contribute a synthetic robustness protocol that isolates the join step under controlled detector noise.

C. VLMs and LLMs for Circuit Understanding

General-purpose VLMs and LLMs have been applied broadly to electronic design [13], and to circuit understanding in particular. Kulkarni *et al.* [3] proposed Near Sight Correction using VLMs, reporting 87.4% accuracy on circuit VQA and 0.87 F1 on connection-matrix prediction. However, multiple independent benchmarks show that general VLMs handle component recognition far better than connectivity. On the DiagramNet connection task [5], leading general models score very low F1 (GPT-5 at 0.029, Claude-Sonnet-4 at 0.265, and Gemini-2.5-Pro at 0.008), whereas a task-tuned 3B model reaches 0.735. AMSBench [7] reports a similar gap for MLLMs on AMS connectivity. This corroborates the design choice, shared with SINA [4], to keep wire-to-component connectivity out of the VLM and instead solve it with an explicit geometric model. We make this separation concrete and quantify it with our own Claude-VLM connectivity experiment.

D. Wire Detection and Graph-Based Connectivity

Classical wire extraction relies on edge detection, morphological operations, and the Hough transform [19], typically preceded by adaptive binarization such as Sauvola thresholding [14] and connected-component labeling [20]. The production join in existing pipelines uses radius-based proximity: each wire endpoint grabs all component pins within a fixed threshold (typically 30px), connected via transitive union-find [21]. This over-merges when multiple components fall within the threshold, shorting two-terminal parts (R, C, L, D) onto the same net. SINA [4] likewise derives nets from CCL proximity, which is similarly vulnerable to over-merge under detector noise. We instead model joining as a typed endpoint graph that incorporates component geometry and directional preference, and resolve residual ambiguity with a degree-budget completion: a bounded b -matching formulation [22], [24] that caps per-pin degree and thereby limits over-merge when endpoints are dropped or displaced. This differs from graph neural networks for netlist reverse engineering [23], which learn representations over already-extracted gate-level graphs; our graph is a purely *geometric* model whose edges encode spatial relationships between wire endpoints and component pins. A complementary recent direction replaces rule-based connectivity with a *learned* model: Netlistify [25] predicts terminal connectivity with a Transformer and reports a

12.4% F1 gain over AMSnet on printed AMS schematics. Such models are powerful but require task-specific connectivity training data and, trained on printed schematics, do not transfer to hand-drawn images without retraining; our geometric model needs no connectivity training and is the focus here. Open hand-drawn efforts such as CircuitNet [26] likewise rely on traditional image processing plus a classifier and a heuristic netlist generator.

III. METHOD

A. Pipeline Overview

The full pipeline proceeds through six stages: (1) component detection via oriented bounding box (OBB) YOLO model; (2) component occlusion (filling detected regions with local median color); (3) Sauvola adaptive binarization; (4) connected component labeling with morphological post-processing; (5) wire endpoint extraction via PCA on endpoint regions; and (6) wire joining via the endpoint-graph model with degree-budget completion. The output is a netlist mapping component pins to electrical nodes, suitable for SPICE netlist generation.

B. Endpoint-Graph Join Model

Given a set of wire segments $\mathcal{W} = \{w_1, \dots, w_m\}$ with endpoints $\text{ep}(w_i) = (a_i, b_i)$ and component pins $\mathcal{P} = \{p_1, \dots, p_n\}$, we construct an undirected graph $G = (V, E)$ where $V = \text{eps}(\mathcal{W}) \cup \mathcal{P}$.

Edge type 1 (wire body): For each wire w_i , add edge (a_i, b_i) with weight 0. This ensures both endpoints of a wire are always in the same net.

Edge type 2 (endpoint–endpoint): For endpoint pairs (e_i, e_j) with $\|e_i - e_j\| \leq \tau_{\text{join}}$, add edge with weight $\|e_i - e_j\|$. This captures wire fragments, corners, and junctions.

Edge type 3 (endpoint–pin): For each endpoint e and pin p with $\|e - p\| \leq \tau_{\text{pin}}$, add edge with weight $\|e - p\| \cdot (1 - \alpha \cdot \cos \theta)$, where θ is the angle between the wire direction and the endpoint-to-pin vector, and α controls directional preference. This is the critical edge type: it connects wires to components.

Edge type 4 (endpoint–wire-body): For each endpoint e and wire segment $w_j = (c, d)$, if the point-to-segment distance $\text{dist}(e, w_j) \leq \tau_t$, add edge with weight $\text{dist}(e, w_j)$. This captures T-junctions where a wire terminates on another wire’s body.

Edge type 5 (pin–wire-body): For each component pin p and wire segment w_j , if the point-to-segment distance $\text{dist}(p, w_j) \leq \tau_t$ and the projection lands mid-span (not at w_j ’s endpoints, which are handled by edge type 3), add edge with weight $\text{dist}(p, w_j)$. This binds components that tap onto a passing rail along the wire’s body rather than at its end. It recovers connectivity that endpoint-only binding misses, without the all-to-all over-merge of radius grabbing.

The tolerances τ_{join} , τ_{pin} , and τ_t are *scale-relative*: $\tau = k \cdot s$ where s is the characteristic scale (median component diagonal), so a single parameter setting works across circuits of varying size.

Component-first pin assignment: For edge type 3, we use a two-step assignment. First, assign each endpoint to its nearest

(a) Radius-based joining (production) (b) Endpoint-graph joining (ours): five typed edges

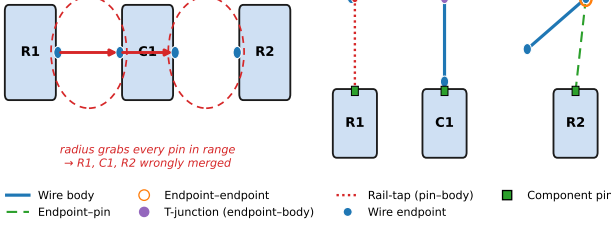


Fig. 3. Joining approaches compared. (a) Production radius-based join: each wire endpoint grabs ALL pins within a fixed threshold, causing over-merging. (b) Our endpoint-graph model: typed edges (wire body, endpoint-pin) with directional preference resolve connectivity without over-merging.

component by bounding-box distance (0 if inside the bbox, using a generous radius of $\max(\tau_{\text{pin}}, 0.5 \cdot \text{diagonal})$). Second, within the assigned component, select the pin by geometric position: for horizontal components ($w > h$), left pin if endpoint $x < \text{center } x$, else right pin; for vertical components ($h > w$), top pin if endpoint $y < \text{center } y$, else bottom pin. This handles the common case where endpoints land inside a component’s bounding box but far from its pin.

Nets are computed as connected components of G , projected onto pins. The result is a *Netlist* mapping each pin to a net identifier.

C. Degree-Budget Completion

The endpoint-graph join may leave some component pins “floating”: touching only their own component’s net, with no wire connecting them to another component. This indicates a dropped or over-displaced wire endpoint that the detector failed to place near the correct pin.

Degree-budget completion identifies these floating pins and reconnects them via constrained optimization:

- 1) **Identify floating pins:** A pin p is floating if its net contains only pins from p ’s own component (excluding inert types: gnd, vss, text, antenna, junction, terminal).
- 2) **Candidate edges:** For each floating pin p , generate candidate edges to all pins q in other components within reach $r = \text{REACH_FACTOR} \cdot s$ (default $\text{REACH_FACTOR} = 2.5$). Edge weight is Euclidean distance $\|p - q\|$.
- 3) **Min-cost b-matching:** Solve $\min \sum_{(p,q) \in E'} w(p,q) \cdot x_{pq}$ subject to $\sum_q x_{pq} \leq 1$ for each floating pin p (at most one connection per pin), using the Hungarian algorithm [24] (`scipy.optimize.linear_sum_assignment`). Each target pin offers up to three slots so legitimate multi-terminal nets are not starved, and a per-pin opt-out leaves a pin with no good target unconnected rather than forcing a spurious edge.
- 4) **Self-loop guard:** Reject any matching edge (p,q) if p and q belong to the same component or if adding the edge would merge two nets that already share a component. This structurally prevents short-circuiting two-terminal parts.

Before completion After completion

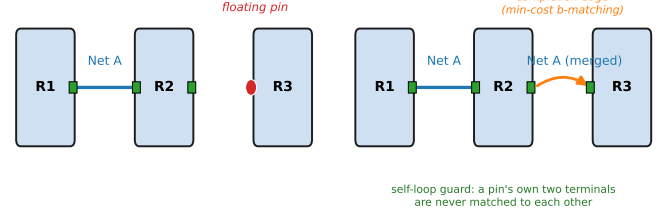


Fig. 4. Degree-budget completion. Left: After base joining, some pins are “floating” (touching only their own component). Right: Completion reconnects floating pins via min-cost b-matching (orange dashed), with self-loop guard preventing shorts within two-terminal components.

Completion edges are “wireless”: they represent inferred connections without corresponding wire segments, analogous to manual pin merges in schematic editors.

IV. SYNTHETIC EVALUATION FRAMEWORK

A. Circuit Corpus

We author 15 synthetic circuits spanning 3–6 components and 2–6 nets, including parallel resistors, series dividers, Wheatstone bridges, RC/RL circuits, diode loops, and a 6-component ring (the hardest topology for join robustness). Each circuit has a known ground-truth netlist and SPICE deck with a test voltage source.

B. Error Injection Model

We inject five categories of error at four severity levels (L1–L4) on top of the clean (L0) baseline:

- **Jitter:** Gaussian endpoint perturbation with standard deviation σ (L1: 3px, L4: 14px)
- **Cut-short:** Each wire shortened inward by a fixed length (L1: 4px, L4: 22px), modeling endpoints that stop short of the pin
- **Drops:** Entire wire removed with probability p_{drop} (L1: 0, L4: 0.20)
- **Wrong-pin:** Endpoint snapped to a wrong nearby pin with probability p_{wp} (L2: 0.02, L4: 0.10)
- **Anchor displacement:** With probability p_{disp} (L1: 0.10, L4: 0.40), one endpoint is displaced by up to a_{max} pixels (L1: 30px, L4: 80px)

C. Metrics

Join F1: Component-connectivity F1 score measuring which pairs of components share a net in the predicted vs. ground-truth netlist.

Simulation accuracy (sim_ok): Percentage of random seeds where every source current in the predicted SPICE netlist matches the clean oracle within 5%.

V. RESULTS

A. Wire Detection Benchmark

We evaluate on 134 images from CGHD-1152 with ground-truth wire and component labels. Table I shows the top configurations across 36 tested variants.

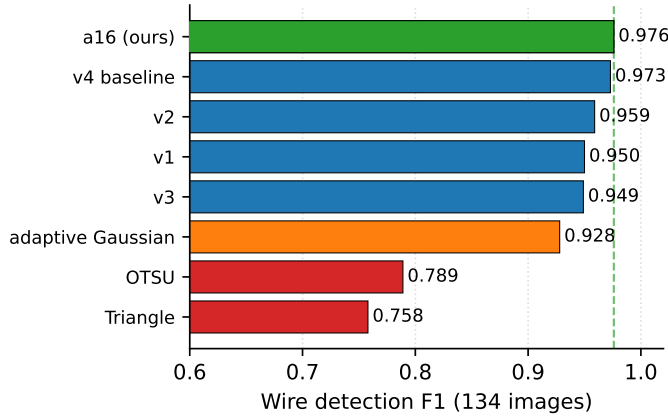


Fig. 5. Wire detection F1 across thresholding methods and pipeline variants (134 images). Our configuration (a16) achieves $F1 = 0.976$. OTSU and triangle thresholding degrade significantly.

TABLE I
WIRE DETECTION PERFORMANCE (134 IMAGES)

Config	F1	Prec	Rec
a16 (Sauvola + anchor=16)	0.976	0.973	0.978
v4 baseline (anchor=12)	0.973	0.974	0.972
best_candidate_v2	0.959	0.944	0.974
best_candidate_v1	0.950	0.921	0.980
OTSU + component	0.789	0.796	0.783

The pipeline achieves median $F1 = 1.000$ across the benchmark set, with 87% of images scoring $F1 \geq 0.90$. Sauvola adaptive binarization dominates all alternatives: OTSU yields $F1 = 0.789$, adaptive Gaussian $F1 = 0.928$. Component occlusion matters: without it, component internals (text, symbols, internal wiring) generate hundreds of false wire detections.

B. Join Strategy Comparison

Table II compares joining strategies on the synthetic evaluation framework (15 circuits $\times$ 8 seeds, mean F1 over error levels L1–L4).

TABLE II
JOIN STRATEGY LEADERBOARD (SYNTHETIC EVAL)

Strategy	L0	L1	L2	L3	L4
scale_completion	1.00	1.00	1.00	0.96	0.95
degree_budget	1.00	1.00	1.00	0.95	0.94
graph_rescue	1.00	1.00	1.00	0.94	0.89
graph_full	1.00	1.00	1.00	0.94	0.89
graph_scale	1.00	1.00	1.00	0.94	0.85
production	1.00	0.97	0.90	0.56	0.36
nearest1_30	1.00	0.97	0.90	0.56	0.36
anchored1_30	1.00	0.97	0.89	0.38	0.32

The completion-based strategies are the most robust, maintaining $F1 \geq 0.94$ at the highest error severity (L4), compared to $F1 = 0.36$ for the legacy radius-based production baseline, a $2.6\times$ improvement. The production join’s transitive radius-based grabbing causes catastrophic over-merging under

noise: at L4, it shorts multiple components onto shared nets. We emphasize, however, that the endpoint-graph baselines (graph_rescue, graph_full) are themselves far more robust than the radius join, reaching $F1 = 0.89$ at L4 (Table II). The degree-budget completion adds a further $+0.05$ over the strongest of these by recovering dropped connections the base graph join leaves floating, while its per-pin edge budget keeps precision high. Pairing that completion with the high-precision scale-relative base (scale_completion) gives the best synthetic robustness (0.95 at L4) and, more importantly, the best *real-image* connectivity (Section V-C). The large gap is against the legacy radius join; against the strongest prior graph join the improvement is smaller but consistent across every severity level.

C. Real-Image Net-Level Evaluation

The synthetic results above measure robustness under a parametric error model. To validate on real detector output, we constructed, to our knowledge, the first *human-verified net-level* ground truth for hand-drawn circuits: 31 images from CGHD-1152 in which every electrical net was traced and confirmed by a human annotator through a purpose-built verification tool. Because one of our comparison points is a vision-language model (Section V-E), the *human*, not any model, is the verifier of record, so the answer key is independent of every method under test. Net-level proposals were bootstrapped by a conservative endpoint-graph join over the dataset’s perfect wire labels and then corrected by hand; during verification the human consistently caught cases where the bootstrap join had wrongly split a single rail, and we additionally re-validate all conclusions on the independently authored synthetic ground truth to rule out residual bootstrap bias. Connectivity is scored as component-pair F1 restricted to SPICE-active components, identical to the synthetic metric.

Table III reports every registered join strategy on the verified set, run on the pipeline’s own detected wires. We report micro-averaged (pair-pooled) component-pair F1 as the primary metric, so that a dense 14-component board contributes in proportion to its connections rather than counting the same as a 3-component one; the macro (mean per-image) F1 is listed alongside for reference. The best configuration, scale_completion (the degree-budget completion applied to a high-precision scale-relative endpoint-graph base), achieves micro-F1 = 0.890 ($P = 0.92$, $R = 0.86$; 95% bootstrap CI [0.855, 0.924]), improving on the previous degree-budget default (0.829) and the radius-based production join (0.667) (Figure 7). The gain over the earlier graph_rescue-based completion comes from the cleaner base: the rescue base’s 12px end-extension and dead-end rescue over-reach on noisy detected wires and cost precision, whereas the scale-relative base keeps precision high while the bounded completion still recovers recall. A reach sweep shows a broad F1 plateau across reach $3\text{--}5 \times s$, so the setting is robust rather than tuned to a spike. Confidence intervals are wide at $N = 31$ and the per-strategy intervals overlap, but the ranking is consistent and is corroborated on the independently authored synthetic suite (Table II).

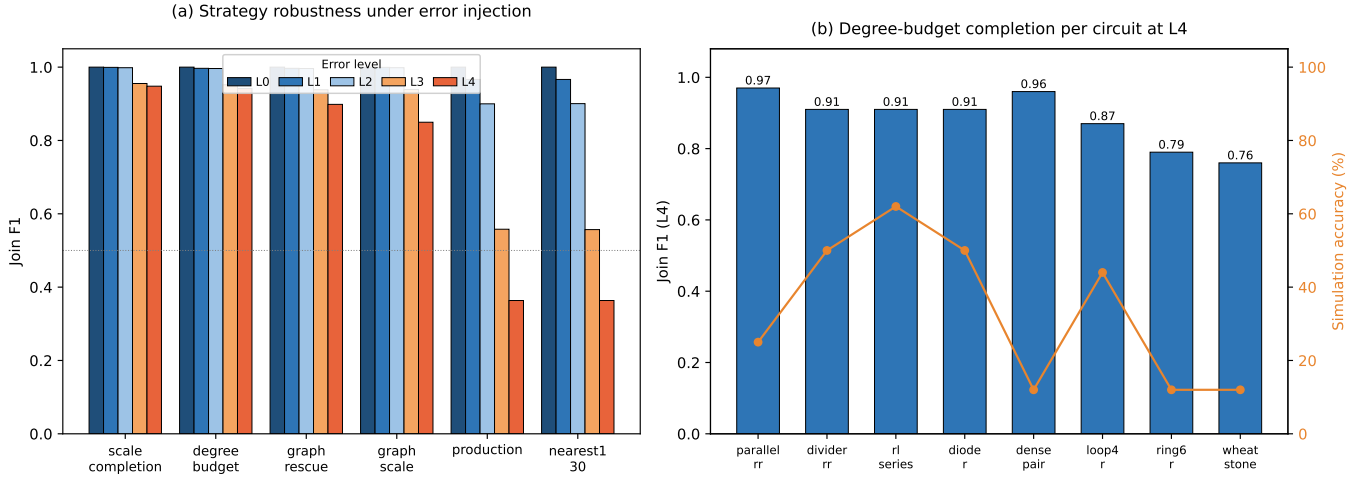


Fig. 6. (a) Join strategy robustness under error injection (15 circuits $\times$ 8 seeds). Graph-based strategies (blue) outperform radius-based approaches (red) at higher error levels. (b) Degree-budget performance per circuit at L4, with simulation accuracy (orange). Precision remains high (≥ 0.85): errors cause fragmentation, not shorts.

TABLE III
JOIN STRATEGY AND BASELINE COMPARISON ON HUMAN-VERIFIED
NET-LEVEL GROUND TRUTH (31 REAL IMAGES, DETECTED WIRES).
PRIMARY METRIC IS MICRO-AVERAGED (PAIR-POOLED) COMPONENT-PAIR
F1; MACRO (MEAN PER-IMAGE) F1 IS SHOWN FOR REFERENCE.

Method	F1	Prec	Rec	F1 _{macro}
<i>Deterministic joins (ours and ablations)</i>				
scale_completion (ours)	0.890	0.919	0.864	0.901
degree_budget (prior default)	0.829	0.789	0.874	0.853
graph_scale	0.816	0.929	0.727	0.838
graph_rescue	0.787	0.811	0.764	0.813
production (radius)	0.667	0.698	0.638	0.674
<i>Classical deterministic baselines (identical detected wires)</i>				
Hough + proximity (best)	0.805	0.750	0.868	0.847
Connected-components (best)	0.624	0.965	0.461	0.611
<i>Reference (non-deterministic / oracle wires)</i>				
VLM (Claude Opus 4.8)	0.923	0.970	0.880	0.949
scale_completion, perfect wires	0.890	0.913	0.868	0.916

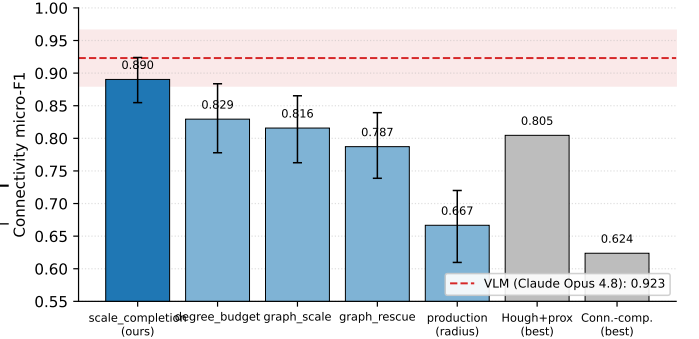


Fig. 7. Micro-averaged connectivity F1 on the 31-image human-verified net-level benchmark (component-pair metric). Our *scale_completion* join leads all deterministic strategies and the classical Hough and connected-component baselines (run on identical detected wires). The dashed line marks a strong VLM evaluated on the same 31 images, from which our join is statistically indistinguishable (paired micro-F1 difference $+0.033$, 95% bootstrap CI $[-0.009, +0.078]$) while being deterministic, simulatable, and orders of magnitude cheaper. Error bars are 95% bootstrap CIs.

This real-image ranking resolves an open question from prior work on this pipeline: against bootstrapped pilot labels the degree-budget completion had *not* consistently beaten its base join; against human-verified ground truth it clearly does (micro-F1 0.829 vs 0.787 for *graph_rescue*), and the scale-relative variant is better still. The same ordering holds on the authored synthetic suite (Table II), where *scale_completion* also ranks first, so the result is not an artifact of how the net-level labels were bootstrapped.

Comparison to connected-component net tracing. The dominant connectivity recipe in prior image-to-netlist systems erases component boxes and reads nets off the connected components (or contours) of the residual wire pixels. It is used by SINA [4], the concurrent contour-based node detector of Peker et al. [9], AMSnet [6], and the Bayer instance-segmentation pipeline [10], and was designed for clean printed schematics. On hand-drawn photographs it has no precise

operating point: pencil-stroke fragmentation splits each net into many components ($F1 < 0.25$ at sensible morphological gap-bridging), and only pathological closing that effectively merges by proximity lifts recall, at a steep precision cost. For a controlled comparison that isolates the join algorithm from binarization quality, we run connected-component labeling on our *own* detected wire segments (identical input to our join), and its best configuration reaches only micro-F1 = 0.624 (recall collapses to 0.46: the blobs short or drop most nets), versus 0.890 for the endpoint-graph join (Table III). A stronger classical alternative, the Hough-plus-proximity line-tracing of Reddy and Panicker [12], fares better, since line fitting denoises the fragmented strokes, and reaches micro-F1 = 0.805 at its best-tuned linking tolerance, but still trails our join by 0.085 at markedly lower precision. (Each baseline is reported at its best configuration over a tolerance sweep, so these are upper bounds on the classical recipes.) Explicit typed-endpoint

reasoning with bounded completion thus outperforms both blob/contour connectivity and line-primitive proximity tracing.

Reproducing prior systems. We additionally attempted to run five public image-to-netlist systems end-to-end on our hand-drawn benchmark; none could be fairly applied. The classical printed-schematic interpreter of Kelly and Cole [2] installs and runs, but on hand-drawn scans it over-segments wires into spurious components and recovers almost no connectivity, a domain-transfer failure. Netlistify [25] reproduces on its own printed AMS test data (we ran its released model end-to-end), but its active model lacks the diode, inductor, and IC classes our circuits contain and its connectivity model is trained on printed thin-line appearance, so it cannot represent hand-drawn discrete circuits. Masala-CHAI [8] delegates connectivity to a proprietary LLM behind a paid API (equivalent in spirit to our VLM experiment, Section V-E) rather than a deterministic algorithm. SINA’s [4] anonymized code archive was unextractable and CircuitNet [26] ships only Colab notebooks without released weights. This survey is itself informative: no prior system provides a deterministic, hand-drawn-ready connectivity module, so the controlled CCL and Hough baselines above, both run on our identical detected wires, are the fair points of comparison, and our join exceeds both.

Detection is not the bottleneck. Running the best join on the dataset’s perfect wire labels rather than detected wires leaves micro-F1 unchanged at 0.890 (it raises macro-F1 only from 0.901 to 0.916; Table III), so the wire detector accounts for essentially zero pair-weighted F1. The residual error is intrinsic join ambiguity on hand-drawn geometry, not wire detection, consistent with the 0.976 wire-detection F1 reported above (Table I). The join degrades only gently with circuit size (per-image F1 falls from 0.94 on circuits with ≤ 6 components to 0.86 above that, weak correlation $r = -0.19$) and remains high-precision throughout, in contrast to the VLM, whose precision collapses on the densest circuits (Section V-E).

D. Per-Circuit Analysis

Table IV shows degree-budget performance across individual circuits at L4 (extreme error injection, 16 seeds).

TABLE IV
DEGREE-BUDGET PERFORMANCE PER CIRCUIT AT L4

Circuit	Comps	F1	Prec	sim_ok
parallel_rr	3	0.97	1.00	25%
divider_rr	3	0.91	0.94	50%
rl_series	3	0.91	1.00	62%
diode_r	3	0.91	1.00	50%
dense_pair	4	0.96	1.00	12%
loop4_r	4	0.87	0.97	44%
wheatstone	6	0.76	0.85	12%
ring6_r	6	0.79	0.97	12%

Precision remains high (≥ 0.85) across all circuits even at L4: errors cause *fragmentation* (dropped connections) rather than *shorts* (false merges). The 6-component ring (ring6_r) is the hardest topology: its cyclic structure means a single

dropped connection breaks the loop, reducing simulation accuracy to 12%.

E. VLM Connectivity Experiment

To quantify the separation argued in Section II-C, that connectivity is better solved by an explicit geometric model than left to a VLM, we ran a first-party experiment with a strong general VLM (Claude Opus 4.8). To make the comparison fair, the VLM is given exactly what the join receives: the scanned image with the same ground-truth component boxes labelled by index, and is asked to return, as JSON, the sets of component terminals that share an electrical node; we score the resulting component-pairs with the same metric. Each image is read by an independent, context-free model call (one per image), so no answer leaks between circuits, and the human, not the model, remains the verifier of record. On a clean synthetic control (15 authored circuits) the VLM scores $F1 = 0.99$, confirming it understands the task and symbol set. On all 31 real scans, evaluated against the human-verified net-level ground truth, it reaches micro-F1 = 0.923 ($P = 0.97$, $R = 0.88$; macro-F1 = 0.949).

The VLM thus edges our deterministic join on aggregate (0.923 vs 0.890 micro-F1), but the gap is not statistically significant: a paired bootstrap over the 31 images puts the difference at +0.033 with a 95% CI of $[-0.009, +0.078]$, which includes zero. Two further findings qualify the comparison. First, the VLM’s accuracy degrades with circuit complexity: it is exact ($F1 = 1.00$) on 20 of the 31 circuits, all of the small 3–5-component cases among them, but on the largest boards it systematically *omits* connections, with recall falling to 0.55–0.68 on the densest circuits (e.g. the 88-, 57-, and 47-pin boards) exactly where a netlist error is most costly; its precision otherwise stays high (0.97 micro), so its failure mode is dropped nets rather than shorts. The macro-F1 (0.949) is correspondingly inflated by the many easy small circuits, which is why we report the pair-weighted micro-F1 as primary. Second, the cost and form of the output differ from the geometric pipeline: each image consumes on the order of 10^5 tokens, the result is free-form text rather than a structured netlist, and nothing in the model guarantees electrical validity (e.g. that a two-terminal component is not shorted). Our geometric join is statistically indistinguishable from the VLM on these circuits while being deterministic, two to three orders of magnitude cheaper, structurally guaranteed against shorts by the self-loop guard, and simulatable by construction. We therefore treat the VLM as an upper reference for what a frontier model can do here, not as a connectivity module to ship: an explicit model reaches the same accuracy and also gives the determinism, low cost, and structural validity a deployed digitizer needs.

VI. DISCUSSION

Why graph-based joining works. The endpoint-graph model resolves ambiguity that radius-based methods cannot: by representing both wire endpoints and component pins as nodes with typed edges, it distinguishes between a wire that *passes near* a component (endpoint–wire-body edge) and one

that *connects to* it (endpoint–pin edge with directional preference). The component-first assignment further disambiguates by using the geometric constraint that wires must reach a component’s pin, not just its bounding box.

Completion as fault recovery. Degree-budget completion addresses the fundamental limitation of detector noise: even a 97.6% accurate wire detector drops $\sim 2\%$ of wire endpoints, and those dropped endpoints are precisely the connections that matter most for simulation accuracy. The b-matching formulation bounds the damage from mis-detection (each floating pin gets at most one recovery edge), while the self-loop guard prevents the catastrophic short-circuit failures that plague unconstrained approaches.

Limitations. The human-verified net-level evaluation, while novel for hand-drawn circuits, currently covers 31 images; broader annotation would tighten the confidence intervals on the strategy ranking, though the ordering is corroborated by the independently authored synthetic suite. The net-level labels were bootstrapped by a conservative endpoint-graph join over perfect wires and then human-corrected; we mitigate the residual concern that this favors graph-based methods by validating on synthetic ground truth that is authored from scratch, where the same ranking holds. The evaluation draws from a single hand-drawn corpus (CGHD-1152); generalization across other drawing styles, capture conditions, and printed schematics remains to be validated, as does extension to circuits containing devices outside the simulatable set (switches, transformers, etc.), which we exclude from the net-level benchmark. The VLM connectivity comparison covers nine images for which responses were collected; a larger VLM evaluation is future work. Component detection at 88.5% mAP50 also leaves room for improvement, particularly on crossovers (70.7% recall). Finally, the synthetic error model is a first-pass approximation of detector noise rather than a calibrated fit; with the new real-image net-level metric available, calibrating it to the measured detector failure distribution is a natural next step.

VII. CONCLUSION

We present a two-stage wire joining approach for circuit diagram digitization and, to our knowledge, the first human-verified net-level connectivity benchmark for hand-drawn circuits. The endpoint-graph join with degree-budget completion is most robust under synthetic error injection ($F1 = 0.95$ at the highest severity vs. 0.36 for the radius baseline), and, paired with a high-precision scale-relative base, attains connectivity $F1 = 0.90$ on real images against human-verified ground truth, beating the prior completion default (0.85) and the connected-component net-tracing recipe used by recent systems (0.61 on identical detected wires). Running the join on perfect wires reaches only 0.92, showing the join, not wire detection, is the bottleneck and that our method nearly closes the gap. A first-party VLM experiment attains comparable accuracy at far higher cost and without simulatable, structurally valid output, supporting the explicit geometric design. Future work includes scaling the human-verified benchmark, calibrating the synthetic error model to the measured detector distribution, and extending coverage to devices outside the simulatable set.

DATA AND CODE AVAILABILITY

The pipeline, evaluation scripts, synthetic-circuit generator, the 31-image human-verified net-level ground truth (ground_truth/real_nets_verified.json), the net-level verification tool used to produce it, the join-strategy and connected-component baseline benchmarks, and the Claude-VLM connectivity harness are available at <https://github.com/boscochanam/circuit-digitization>. The component-detection model is available at <https://huggingface.co/boscochanam/circuit-component-detector>, trained on the CGHD-1152 dataset [1].

REFERENCES

- [1] J. Bayer, “CGHD-1152: Circuit graph hand-drawing dataset,” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/johannesbayer/cghd1152>
- [2] C. R. Kelly and J. M. Cole, “Digitizing images of electrical-circuit schematics,” *APL Mach. Learn.*, vol. 2, no. 1, p. 016109, 2024.
- [3] S. Kulkarni *et al.*, “Enhancing circuit diagram understanding via near sight correction using VLMs,” in *Proc. ICCV Workshops*, 2025.
- [4] S. Aldowaihi, Y. Karumanchi, K.-C. Chiang, S. Noorzad, and M. Fayazi, “SINA: A circuit schematic image-to-netlist generator using artificial intelligence,” arXiv:2601.22114, 2026.
- [5] J. Lou, R. Xu, J. Li, J. Pi, R. Tao, W. Fan, X. Tan, G. Luo, and Y. Lin, “DiagramNet: An end-to-end recognition framework and dataset for non-standard system-level diagrams,” arXiv:2605.01338, 2026.
- [6] Y. Shi, Z. Tao, Y. Gao, L. Huang, H. Wang, Z. Yu, T.-J. Lin, and L. He, “AMSnet 2.0: A large AMS database with AI segmentation for net detection,” arXiv:2505.09155, 2025.
- [7] Y. Shi, Z. Zhang, H. Wang, Z. Tao, Z. Li, B. Chen, Y. Wang, Z. Huang, X. Liu, Q. Chen, Z. Yu, T.-J. Lin, and L. He, “AMSbench: A comprehensive benchmark for evaluating MLLM capabilities in AMS circuits,” arXiv:2505.24138, 2025.
- [8] J. Bhandari, V. Bhat, Y. He, H. Rahmani, S. Garg, and R. Karri, “Masala-CHAI: A large-scale SPICE netlist dataset for analog circuits by harnessing AI,” arXiv:2411.14299, 2024.
- [9] Ö. B. Peker, E. Toker, D. Öcal, T. Dalyan, E. Afacan, and Y. D. Gökdel, “A fully automated SPICE-compatible netlist extraction from image using deep learning and image preprocessing techniques,” *IEEE Access*, vol. 14, pp. 19750–19765, 2026, doi: 10.1109/ACCESS.2026.3656316.
- [10] J. Bayer *et al.*, “Instance segmentation based graph extraction for handwritten circuit diagram images,” arXiv:2301.03155, 2023.
- [11] A. Roy, S. Pani, S. Malakar, and R. Sarkar, “JUHCCR-v1: A database for hand-drawn electrical and electronics circuit component recognition,” *Sci. Rep.*, vol. 15, art. no. 22404, 2025, doi: 10.1038/s41598-025-22404-5.
- [12] R. R. Reddy and M. R. Panicker, “Hand-drawn electrical circuit recognition using object detection and node recognition,” *SN Comput. Sci.*, vol. 3, no. 3, p. 244, 2022. arXiv:2106.11559.
- [13] R. Zhong, X. Du, S. Kai, Z. Tang, S. Xu, H.-L. Zhen, J. Hao, Q. Xu, M. Yuan, and J. Yan, “LLM4EDA: Emerging progress in large language models for electronic design automation,” arXiv:2401.12224, 2024.
- [14] J. Sauvola and M. Pietikäinen, “Adaptive document image binarization,” *Pattern Recognit.*, vol. 33, no. 2, pp. 225–236, 2000.
- [15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [16] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [17] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [18] L. W. Nagel and D. O. Pederson, “SPICE: Simulation program with integrated circuit emphasis,” Electron. Res. Lab., Univ. California, Berkeley, Memo. ERL-M382, 1973.
- [19] R. O. Duda and P. E. Hart, “Use of the Hough transformation to detect lines and curves in pictures,” *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.
- [20] L. He, X. Ren, Q. Gao, X. Zhao, B. Yao, and Y. Chao, “The connected-component labeling problem: A review of state-of-the-art algorithms,” *Pattern Recognit.*, vol. 70, pp. 25–43, 2017.
- [21] R. E. Tarjan, “Efficiency of a good but not linear set union algorithm,” *J. ACM*, vol. 22, no. 2, pp. 215–225, 1975.

- [22] A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency*. Berlin, Germany: Springer, 2003.
- [23] L. Alrahis, A. Sengupta, J. Knechtel, S. S. Gharibeh, O. S. Unsal, M. E. Acar, and H. Najjar, “GNN-RE: Graph neural networks for reverse engineering of gate-level netlists,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2021, pp. 1–9.
- [24] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [25] C.-Y. Huang, H.-I. Chen, H.-W. Ho, P.-H. Kang, M. P.-H. Lin, W.-H. Liu, and H. Ren, “Netlistify: Transforming circuit schematics into netlists with deep learning,” in *Proc. ACM/IEEE Symp. Machine Learning for CAD (MLCAD)*, 2025.
- [26] “CircuitNet: A hand-drawn schematic sketch recognizer and converter,” GitHub repository (user aaanthonyyy), 2024. [Online]. Available: <https://github.com/aaanthonyyy/CircuitNet>

Bosco Chanam received the [degree] from [institution]. His research interests include computer vision, document understanding, and circuit digitization. [Fill before submission.]

Chris Dcosta received the [degree] from [institution]. His research interests include [fill]. [Fill before submission.]

Pranavesh Talupuri received the [degree] from [institution]. His research interests include [fill]. [Fill before submission.]